

>

HannesF99

Aug 22 '16

I have a code like
myKernel<<<...>>>(srcimg, dstimg)
cudaMemcpy2D(..., cudaMemcpyDeviceToHost)
where the CUDA kernel computes an image 'dstimg' (dstimg has its buffer in GPU memory) and the cudaMemcpy2D fn. then copies the image 'dstimg' to an image 'dstimgCpu' (which has its buffer in CPU memory).
Do I have to insert a 'cudaDeviceSynchronize' before the 'cudaMemcpy2D' in order to ensure that the kernel has finished writing the values to 'dstimg' GPU memory buffer ? From [cuda - When to call cudaMemcpySynchronize? - Stack Overflow](#) it looks like I do NOT have to insert the 'cudaDeviceSynchronize' after the kernel, but I am not 100% sure.

Robert_Crovella

Moderator

Aug 22 '16

CUDA operations issued to the same stream *always* serialize.
Assuming your kernel is issued to the default stream, there is no possibility for a cudaMemcpy2D operation to begin before that kernel has completed.
All global memory updates made by a kernel are guaranteed to be visible to any actor, when the kernel completes.
Assuming all these operations are to the default stream, there is no need for a cudaDeviceSynchronize() after a kernel launch, before a cudaMemcpy (or cudaMemcpy2D) operation.

HannesF99

Aug 26 '16

thx for the info. What if 'myKernel' is not launched on the default stream (which has a special behaviour) ?

tera Top Contributor

Aug 26 '16

As long as kernel and cudaMemcpy() are issued on the same stream (whether default or not), no synchronization is necessary. **Operations within the same stream are always executed in order** , only operations from different streams may execute in parallel.

HannesF99

Aug 26 '16

But 'cudaMemcpy' (and 'cudaMemcpy2D') does not have a 'stream' parameter, only the 'async' variants have ...

Robert_Crovella

Moderator

Aug 26 '16

Correct. So it is not possible to issue them to anything other than the default stream. A cudaMemcpy operation is always issued to the default stream.
Since you didn't provide anything that looked like actual code, it's always possible that someone says "my code is like ... cudaMemcpy" when really what they mean is "I am actually using cudaMemcpyAsync"
So my answer was a general answer, and some of your subsequent questions are answered by my very first statement:
"CUDA operations issued to the same stream *always* serialize."
If, on the other hand, a cuda kernel is launched into one stream and a cudaMemcpyAsync operation is issued into another stream (assuming neither is the default stream with default semantics) then there are no guarantees about the order of execution.

Abbyluo

Dec 28 '16

Hi
I'm confused with the cudaMemcpy2D, as i want to copy a matrix of 20*4 form device to host,but the data is not stable, sometimes one row is zero after copy, my code is like:

```
void region_search()  
{  
    dim3 threadsPerBlock(32, 8);  
    dim3 Grid((512+threadsPerBlock.x-1)/threads  
  
    int *d_num, h_num;  
    unsigned int *d_output_matrix;  
    size_t pitch;  
    cudaMalloc((void**)&d_num,sizeof(int));  
    cudaMallocPitch((void**)&d_output_matrix, &  
    region_search_kernel<<< Grid, threadsPerBloc  
    cudaMemcpy(&h_num,d_num,sizeof(int),cudaMem  
    unsigned int h_output[h_num][4];  
    cudaMemcpy2D(h_output,4*sizeof(unsigned in  
  
    for (int x=0;x<h_num;x++)  
    {  
        printf("output after copy %d %d %d %d\n  
    }  
    cudaFree(d_num);  
    cudaFree(d_output_matrix);  
    free(h_num);  
    free(h_output1);  
}
```

```
__global__ void region_search_kernel(int *d_num  
{  
    int i = threadIdx.x + blockIdx.x * blockDim  
    int j = threadIdx.y + blockIdx.y * blockDim  
    int index_s=threadIdx.x+threadIdx.y*32;//ind  
    int id_block=blockIdx.y*16+blockIdx.x;  
    __shared__ unsigned int data_x[32*8];  
    __shared__ unsigned char data_y[32*8];  
    data_x[index_s]=100;//initialize the array i  
    data_y[index_s]=100;  
    __syncthreads();  
    //there ignore some unimportant code  
    if ( j < 424 )  
    {  
        if ( i < 512 )  
        {  
            //there ignore some code to compute  
            if (index_s==0)  
            {  
                if (data_x[index_s]<100)  
                {  
                    unsigned int * row=(unsig  
                    row[0]=data_x[0]+(blockId  
                    row[1]=data_x[num_[0]-1]+  
                    row[2]=data_y[0]+blockIdx  
                    row[3]=data_y[num_[0]-1]+  
                    printf("output in kernel:  
                }  
            }  
        }  
    }  
}
```

my problem is: most of time the printf are the same, and sometimes the printf in the host have a row of 0!
such as this:
you can only see the last 4 elements and the rows maybe not the same order.
normal as followed:

```
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output after copy 0 223 223 255 255//correspon  
output after copy 1 224 254 253 255//correspon  
output after copy 2 217 223 256 263//correspon  
output after copy 3 224 255 256 263//correspon  
output after copy 4 256 257 257 263//correspon  
output after copy 5 216 223 264 271//correspon  
output after copy 6 224 255 264 271//correspon  
output after copy 7 256 257 264 271//correspon  
output after copy 8 217 223 272 274//correspon  
output after copy 9 224 254 272 279//correspon  
output after copy 10 228 248 280 287//correspon
```

```
unnormal:  
  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output in kernel::id block x_min x_max y_min y_r  
output after copy 0 223 223 255 255//correspon  
output after copy 1 224 254 253 255//correspon  
output after copy 2 217 223 256 263//correspon  
output after copy 3 224 255 256 263//correspon  
output after copy 4 256 257 257 263//correspon  
output after copy 5 224 255 264 271//correspon  
output after copy 6 216 223 264 271//correspon  
output after copy 7 256 257 264 269//correspon  
output after copy 8 0 0 0 0//?????????????????  
output after copy 9 217 254 272 279//correspon  
output after copy 10 227 249 280 287//correspon
```

at the beginning i think it's the synchronization between blocks, but when i add "cudaDeviceSynchronize()"after the kernel, the problem also occurred.
Then,i just use 1D array to copy the data to the host, the problem also occurred.
Last, i add some printf in the end of the kernel,it need some time,the problem disappeared!
So,i think it's the synchronize problem but i don't why i add cudaDeviceSynchronize() failed. and i don't want to spent more time,so, i wonder what's the problem? how to resolved it?

Robert_Crovella

Moderator

Dec 28 '16

Try running your code with cuda-memcheck and also with cuda-memcheck --tool racecheck

If there are any problems reported, you'll need to sort those out.

Abbyluo

Dec 29 '16

Hi,
But i don't know how to use the cuda-memcheck in NVIDIA TX1! As i installed cuda by native!

Abbyluo

Dec 29 '16

OK, after i add the /usr/local/cuda/bin to the PATH i can use the cuda-gdb and cuda-memcheck!
and when i use the cuda-memcheck and cuda-memcheck --tool racecheck there have no problem, and the phenomenon i mentioned above disappeared. so strange!

Robert_Crovella

Moderator

Dec 29 '16

If the problem changes when run under cuda-memcheck, it may still be a race condition. For example you may have a race condition in global memory, which cuda-memcheck will not be able to detect. It's impossible to tell since you've chopped out a bunch of your code.

I think its safe to say you have a coding defect, possibly a race condition, that you will need to debug.
It's not clear to me why you posted on this thread or believe that it may involve host synchronization, but for code launched to the same stream (as yours appears to be), no additional synchronization is needed between a kernel call and a cudaMemcpy operation after the kernel call, as already stated several times in this thread.

tushar.k938

Sep 24 '17

Hi, I am doing device to device cudaMemcpy. In my code I have created two threads.
Thread 1 - Copies data from device memroy to another device memory

Thread 2 - Operates on this copied memory:

On CPU program, how should I come to know that Thread-1 has completed the memcpy job before I instruct another thread to process on the latest data and not on the previously holded data / junk data in the buffer.

As per [CUDA Driver API :: CUDA Toolkit Documentation](#) , "For transfers from device memory to device memory, no host-side synchronization is performed.", so can you please help me in understanding how to handle this situation. If you point out to any reference code, that would be helpful.

Thanks,
Tushar

Robert_Crovella Moderator

Sep 24 '17

Issue those operations in the same thread, to the same stream. Even though no host side synchronization is performed, operations issued in the same stream will always serialize

Otherwise you'll need to use inter-thread synchronization, which is a function of your threading system and not CUDA.

tushar.k938

Sep 24 '17

Thanks txbob for reply, but I still do not understand how would I know that the job is completed by first thread before I signal another thread for further operation. Is there any call back or event I would get after device to device cudaMemcpy gets completed? do you know any such test example (multi-threaded + device to device memcpy) which can be referred to?

Robert_Crovella Moderator

Sep 24 '17

Issue a cuda event into the same stream where you issued the cuda memcpy, then issue a cuda stream wait event operation. That will indicate completion. Or you could use a stream callback.

If it were me, I would put the operations in the same stream. Simple. Problem solved.

Since your copy is from device memory to device memory, I assume the operation you are waiting to issue is a kernel call. Processing data in host code couldn't depend on a device->device memory copy.

tushar.k938

Sep 29 '17

Thanks txbob for your inputs. It worked with your suggestion. :)

New & Unread Topics

Topic	Replies	Views	Activity
Questions about "globaltimer", functionality, accessing and configuring	3	12	16m
Program stucks on cudaErrorMemoryAllocation after failing a cudaMallocHost	12	15	2h
How to operate irregular gemm on tensor core?	8	8	5h
Delay before kernel runs?	6	24	9h
Why Are My CUDA Compilation Results Different with Direct and Separate Compilation?	4	11	9h
Why early exit will influence later __syncthreads()?	2	7	11h
Does TMA need to commit?	0	6	11h
Why will different numbers of thread blocks cause bank conflicts? cuda kernel ubuntu	1	13	Aug 22
CUDA Trigonometric Function Issues	1	13	Aug 22
Is VBIOS version significant to report for reproducibility? pytorch	1	11	Aug 22

There are 198 new topics remaining, or browse other topics in